

Static Reconstruction of an Apple Bootloader Image

iboot_dec_24a435.bin

Binary analysis report

September 13, 2026

Abstract

The supplied 3,832,328-byte image identifies itself as `iBoot for v59, mBoot-20457.2.37`, and `RELEASE`. This report reconstructs its startup mechanisms, selected boot-control paths, command metadata, and subsystem organization from local bytes and AArch64 disassembly. Supporting artifacts contain a reproducible analyzer, a linear disassembly of the apparent primary code region, branch candidates, string references, and command-handler candidates. **This is a partial static reconstruction, not recovered original source, a complete decompilation, or an executable replacement.** No device execution, firmware modification, or security-policy bypass was performed.

Contents

1	Scope, provenance, and evidence standards	2
2	Identity and memory layout	2
3	Startup reconstruction	3
4	Boot coordination and recovery	4
5	Recovered command metadata	5
6	Subsystem reconstruction	6
7	Reconstructed pseudocode	7
8	Reproducibility and coverage	7
9	What is still required for a full reconstruction	8

1 Scope, provenance, and evidence standards

The only executable input examined is `iboot_dec_24a435.bin`. Its SHA-256 digest is:

```
e1b9aa20c3d00b7eeadecfb1dd04963de9a80
e75830cbc816ede8aed000a0301
```

The filename’s `24a435` component is not independently authenticated as an operating-system build number. Likewise, the word `dec` in the filename is not proof of its extraction or decryption history. The file begins with instructions rather than an ELF or Mach-O header. Its readable code and data are consistent with a raw unpacked bootloader payload.

All hexadecimal locations in this report are *file offsets*, except where explicitly called physical or linked addresses. Evidence is classified as:

Observed: literal bytes, decoded instructions, or an exact branch target.

Corroborated inference: a role supported by code, references, and repeated metadata patterns.

Unresolved: a plausible role without a completed control-flow or data-flow proof.

Names such as “permission gate,” “boot coordinator,” and `sub_00081434` are analytical labels, not original symbols. Strings alone do not establish that a feature is enabled or reachable. Apple’s general description of iBoot’s secure-boot responsibilities provides context, not proof of this particular build’s behavior [1].

2 Identity and memory layout

Location	Observation	Interpretation
0x00000000	AArch64 instructions	Entry-like machine setup; little-endian instruction words.
0x00000280	iBoot for v59	Banner also contains Apple copyright years 2007–2026.
0x000002c0	RELEASE	Embedded build-style string.
0x00000300	mBoot-20457.2.37	Embedded build identifier, also referenced by banner code.
0x00000380	0x1fc080000	Physical relocation destination candidate, loaded at entry.
0x00000408	0xfffffc0000000000	Address-bias literal used in startup address arithmetic.
0x00219cd8	Structured pointer data	Apparent end of the main contiguous code region.
0x002b5200 vicinity	Diagnostic strings	Large readable pool, followed by further structured and opaque data.
0x00358000 onward	Data and strings	Secondary metadata/string area; this is not established as executable text.

Location	Observation	Interpretation
0x003a7a08	End of supplied file	Exclusive byte endpoint.

The linked-address candidate is 0xfffffc01fc080000. Subtracting it from many 64-bit pointers gives valid offsets in this image, including pointers to the command names and startup routines. For such pointers,

$$\text{file offset} = \text{linked address} - \text{linked base}.$$

This rule is not applied blindly to hardware addresses or encoded function references. In particular, metadata words beginning with 80f0 or 8620 are retained as encoded values rather than dereferenced as ordinary virtual addresses.

2.1 Zeroed regions and the file-length discrepancy

The literal pair at 0x390/0x398 corresponds to image-relative 0x355908–0x358000. Startup clears this region, and its stored bytes are zero. Another pair at 0x3b8/0x3c0 identifies 0x3a7a40–0x3cddd0, beyond the stored file; this is consistent with zero-initialized runtime storage. The rounded relocation extent 0x3a7a40 is 56 bytes beyond the file’s 0x3a7a08 endpoint. Loader-supplied padding or alignment could explain this, but the required initial memory contents are not recoverable from the file alone. The image must not be treated as a complete runtime memory dump.

3 Startup reconstruction

3.1 Entry, relocation, and overlap handling

The initial sequence writes VTTBR_EL2 and updates HCR_EL2, with synchronization barriers between stages. At 0x1c, AD RP/ADD derives the current image location; 0x24 loads the preferred destination. A direct call at 0x28 targets 0x3b338, a platform helper whose complete semantics remain unresolved. The comparison at 0x2c branches to 0x18c when relocation is unnecessary.

When relocation is needed, the code computes a 64-byte-aligned copy length from header literals. Address comparisons at 0x68–0x84 select safe handling for overlapping ranges. A small copy routine is staged in scratch memory before execution transfers to it:

- 0xf0–0x124: forward copy, 64 bytes per iteration.
- 0x128–0x15c: reverse copy using pre-decremented end pointers.
- 0x160–0x174: erase a source-region remainder in 64-byte chunks.
- 0x178–0x184: erase the staged scratch code in 16-byte chunks.
- 0x188: indirect transfer to the destination image.

Cache invalidation and synchronization appear around the copy routines. This is more specific than a generic “load firmware” operation: the entry actively relocates itself and removes selected temporary copies.

3.2 Runtime establishment

At 0x18c, the code masks interrupts, writes an implementation-specific system register, and sets VBAR_EL1 from an address corresponding to offset 0x1b6800. It clears three literal-delimited memory ranges, then branches from 0x1f8 to 0x33180. That continuation resets TTBR0_EL1/TTBR1_EL1, establishes a stack, calls several platform routines, and performs further processor configuration. The helpers implement additional mapping and initialization; the full hardware register model has not been recovered.

The sequence at 0x33248–0x33280 is an initializer-dispatch loop: it reads 32-bit entries, adds a base, applies PACIZA, and calls through BLRAAZ. Crucially, the start and end literals at 0x33288/0x33290 are *equal* in this image, so that loop executes zero callbacks for these stored values. Its existence does not imply that initializers actually run. The next direct transfer is 0x33284 → 0x1bdb98; the intervening runtime-to-boot initialization chain has not been fully reconstructed.

4 Boot coordination and recovery

The routine near 0x4080 is a strong boot-coordinator candidate. It references the banner, board/revision format, build tag, build style, and serial-number format. It also references recovery settings and directly calls the autoboot routine at 0x486c → 0x4e6c. The following anchors establish actual code references rather than merely words found in the binary:

Code offset	String offset	Referenced text
0x47fc	0x2b55d0	boot-command
0x4804	0x2b5386	device-recovery
0x4890	0x2b5445	delay-recovery-image
0x4974	0x2b546a	Entering recovery mode, starting command prompt
0x4d18	0x2b5661	recover-once
0x4d48	0x2b5646	one-time-boot-command
0x4ed4	0x2b55fe	auto-boot
0x4f94	0x2b5610	bootdelay
0x5018	0x2b561a	aborting autoboot due to user intervention

A conservative behavioral model is: establish the runtime and device state; consult boot/recovery settings; attempt the configured boot path when allowed; otherwise enter the recovery-command environment. This is a high-level inference, not a proof of the ordering of every policy check. In particular, one-shot variables, upgrade handling, and failure fallback precedence require further tracing.

4.1 Autoboot countdown: a verified local behavior

Within the path that reaches 0x4fb8, the returned bootdelay value is retained and multiplied by 1,000,000 using 32-bit arithmetic. A polling loop calls 0x7fd28, tests the result for 13 or 10 (carriage return or line feed), compares an elapsed counter against the threshold, and calls 0x1b65e0 with 10,000 between polls. The counter and delay units are not established here, although the constants

are consistent with microsecond-based timing. On either tested key value, the loop prints the intervention message and leaves the countdown path. The timeout comparison is unsigned and strict `greater-than`.

5 Recovered command metadata

The image contains repeated records with duplicate command-name pointers, a common descriptor pointer, and an encoded handler word. For example, `0x21b388` references `bootx`; its encoded word is `0x80f000000000121c`, whose low 32 bits identify plausible handler code at `0x121c`. Adjacent records commonly have a stride of 120 bytes. The `go` record uses a different upper encoding, `0x86200000000023b0`; its low-bit target is corroborated by memory-image validation and permission-failure code.

The table is a *candidate inventory*, not a proof that all names are exposed in every boot mode, nor a complete decode of the record ABI.

Name	Record offset	Handler offset
<code>bootx</code>	<code>0021b388</code>	<code>0000121c</code>
<code>memboot</code>	<code>0021b400</code>	<code>00001944</code>
<code>go</code>	<code>0021b478</code>	<code>000023b0</code>
<code>reboot</code>	<code>0021c608</code>	<code>00005628</code>
<code>reset</code>	<code>0021c680</code>	<code>00005628</code>
<code>bgcolor</code>	<code>0021c6f8</code>	<code>00005670</code>
<code>setpicture</code>	<code>0021c770</code>	<code>0000570c</code>
<code>firmware</code>	<code>0021c7e8</code>	<code>00005500</code>
<code>devicetree</code>	<code>0021c860</code>	<code>00005af8</code>
<code>getenv</code>	<code>0021c8d8</code>	<code>00162438</code>
<code>saveenv</code>	<code>0021c950</code>	<code>00005624</code>
<code>setenv</code>	<code>0021c9c8</code>	<code>00162530</code>
<code>setenvnp</code>	<code>0021ca40</code>	<code>00162758</code>
<code>clearenvp</code>	<code>0021cab8</code>	<code>001627c0</code>
<code>ramdisk</code>	<code>0021cb30</code>	<code>00005b88</code>

5.1 Handlers with code-level corroboration

memboot: the handler reads `filesize` using a helper at `0x16190c`. Zero produces the invalid-or-missing diagnostic. An unsigned comparison rejects sizes at or above `0x20000001`, so the accepted nonzero size range at this local check is 1–536,870,912 bytes (512 MiB maximum). It then calls `0x81434` and tests result bit 0; a clear bit leads to “Permission Denied.” Subsequent comparisons verify that the requested span fits within returned bounds before further processing. These are observed local checks, not a complete verification policy.

bootx: the handler calls the same gate and has a matching permission-denied branch. It calls the BootKC-related routine at `0x29220`, follows multiple negative-result branches, and updates `boot-breadcrumbs`. The final transfer mechanism and all image validation steps are not fully reconstructed.

go: the handler calls 0x81434, then a separate routine at 0xc6508. A nonzero result from the latter reaches “Memory image not valid”; permission failure is a distinct branch. Thus the name **go** is not evidence of an unrestricted jump primitive.

firmware: the default-argument path checks **filesize** and calls the shared gate before continuing. Its alternate-argument path uses other helpers and has not been fully characterized.

reboot/reset: both records point to 0x5628, establishing an alias at the candidate-handler level.
saveenv: 0x5624 is a one-instruction branch to 0x1623f4, not a standalone persistence implementation.

Environment commands: **getenv**, **setenv**, **setenvnp**, and **clearenvp** lead into the 0x162000 region. Names suggest environment management, but exact persistent versus nonpersistent semantics are not established merely by their suffixes.

Display/ramdisk commands: **bgcolor**, **setpicture**, **devicetree**, and **ramdisk** have distinct candidate handlers. The **setpicture** usage strings describe memory, image-type, and file-based inputs and masks for update, preservation, restore mode, and flipbook selection. The image additionally contains corruption and permission diagnostics referenced by that handler.

6 Subsystem reconstruction

Subsystem	Local evidence	Supported conclusion and limit
Boot objects	BootKC-* references at 0x2a808–0x2a900; BootKC at 0x293fc	Separate labels for read-only, executable, writable, virtual-address, and entry metadata. Exact object layout remains unresolved.
Manifests	0x6ae0 references “verify boot manifest hash”; 0x1f9c90 references a manifest-decode error	Manifest handling exists. The latter is not independently identified as the secure-boot verifier; subsystem ownership matters.
Secure payloads	Names for SecurePageTableMonitor, TrustedExecutionMonitor, cL4, SEP firmware, and trust caches near 0x2b5ac4	Loadable-payload vocabulary is present. This does not prove all payloads are used on v59.
SPTM layout	0x2f5d0 and 0x304c4 reference <code>lay_out_sptm_objects</code>	Code-level anchors for a secure-page-table-monitor layout path. Full semantics are not recovered.
Storage	0x76fe8 references <code>nvme_nand%d</code> ; additional firmware/syscfg device names	NVMe-oriented storage naming and initialization are represented. A full filesystem or controller implementation has not been reconstructed.

Subsystem	Local evidence	Supported conclusion and limit
USB recovery	0xc8d1c references “usb rpc task”; recovery-device and USB-serial strings	USB task/recovery infrastructure is present. The protocol, packet formats, and exposure conditions remain unresolved.
Memory training	0x53ae4 references memory CA calibration failure; extensive Vref and channel diagnostics	Platform memory-calibration code is represented; no full DRAM training algorithm is claimed.
Cryptographic helpers	0x88e04/0x88e14 reference AES argument/key-size errors	AES-related argument validation is present. Algorithms, modes, keys, and secure-boot binding have not been fully traced.
Runtime checks	0x1f54f4 references <code>__firebloom_panic</code> ; pointer bounds comparisons and authenticated returns	Evidence of runtime checks and pointer authentication in sampled code, not a proof of memory safety or exploitability.
Power and diagnostics	Battery state, voltage, panic log, and poweroff strings with platform metadata	Relevant support paths are present; hardware effects require further reverse engineering.

7 Reconstructed pseudocode

The accompanying `reconstruction/behavior.pseudo` expresses only three bounded observations: relocation stages, the `memboot` validation prefix, and the countdown loop. Calls retain offset-based names when their semantics are not proven. It intentionally stops before unresolved processing rather than inventing parsers, cryptographic checks, or a kernel handoff.

8 Reproducibility and coverage

Run the analyzer from the workspace directory:

```
python reconstruct.py
latexmk -pdf -interaction=nonstopmode -halt-on-error main.tex
```

The analyzer uses Python’s standard library and the installed LLVM 19 shared library through its C disassembler interface [2]. No additional packages or virtual environment are needed. Its input hash is checked before analysis. Address bases, code boundary, and command encodings are explicitly image-specific, not a universal iBoot parser.

Inventory metric	Count
strings count	10,182
aligned image pointer count	53,200
candidate start count	17,800
direct call count	101,969
string xref count	2,530
command candidate count	15
undecoded words	692

These numbers are *scan results*, not numbers of verified functions or reachable instructions. Printable runs include accidental matches inside code and opaque data. Candidate starts combine direct-call targets and pointer-authentication prologues, including thunks and internal entries. The 692 undecoded words include literal pools or unsupported encodings; not all represent missing instructions. Conversely, successful decoding alone does not prove that a word is executable.

8.1 Artifact guide

linear_disassembly.txt.gz: AArch64 linear sweep through 0x219cd8, with candidate labels and selected annotations. Embedded header/data bytes are included and must not be mistaken for code.

selected_disassembly.txt: Readable slices for startup, boot coordination, countdown, and candidate command handlers.

commands.json: Raw encoded handler values, name locations, metadata record locations, and inferred target offsets.

branches.json: Direct branch/call candidates; indirect calls are not resolved. “Nearest candidate” is proximity, not function ownership.

string_xrefs.json, address_xrefs.json: Short-window ADRP/ADD matches, not full data-flow analysis.

pointers.json, strings.tsv: Aligned linked-pointer matches and printable byte runs across the whole image.

candidate_starts.json, entropy.json: Entry hypotheses and 64-KiB block statistics. Entropy is not a compression or encryption classifier.

All listed artifacts are under `reconstruction/`. The analyzer is `reconstruct.py`; the report source is `main.tex`.

9 What is still required for a full reconstruction

A complete behavioral reconstruction would require verified function boundaries, indirect-call resolution, structure/type recovery, and path-sensitive analysis of the boot-policy and image-authentication routines. The opaque middle payloads and possible embedded firmware require

separate format identification. The present sweep does not disassemble them as independent processors' firmware.

The key next targets are the shared gate `0x81434`, image-processing routine `0xc6508`, the environment/command dispatcher, the runtime bridge from `0x1bdb98`, and the actual kernel or monitor handoff. Calling conventions must be recovered before replacing the multi-register pointer representations with ordinary C pointers. Validation against an appropriate hardware or emulation environment would then be necessary; no such validation was performed here.

Bottom line: the supplied bytes support a substantial indexed static-analysis corpus and several concrete local reconstructions. They do not yet support claiming a full source recovery, a verified clone, or a complete account of this binary's behavior.

References

- [1] Apple, *Apple Platform Security: Boot process for a Mac with Apple silicon*, published February 18, 2021; consulted September 13, 2026. General platform context only, not identification of the supplied board.
- [2] LLVM Project, *Disassembler.h: C Disassembler Interface*, `LLVMCreateDisasmCPUFeatures`, `LLVMDisasmInstruction`, and `LLVMDisasmDispose`; consulted September 13, 2026. Local implementation: installed LLVM 19 library.